

Содержание

Введение.....	3
1. Анализ предметной области	5
1.1. Нормативная основа и постановка проблемы.....	5
1.2. Обзор методов и аналогов.....	6
1.3. Информационно-эмпирическая база.....	7
2. Проектирование системы	9
2.1. Бизнес- и функциональные требования.....	9
2.2. Ограничения и допущения.....	10
2.3. Архитектура.....	11
2.4. Выбор модели и трекера.....	13
2.5. Определение стоп-линии.....	13
2.6. Определение сигнала светофора	14
2.7. Анализ фотографий.....	15
2.8. Анализ видео	16
2.9. Хранение данных и безопасность	17
3. Реализация и тестирование	18
3.1. Программная реализация	18
3.2. Отчеты и способы запуска	19
3.3. Тестирование	20
3.4. Результаты практической деятельности	21
Заключение	23
Список использованных источников	24
Приложение А. API.....	26
Приложение Б. Установка и использование	27
Приложение В. Репозиторий и демонстрационные материалы	28

Введение

Контроль за соблюдением требования остановки перед стоп-линией при запрещающем сигнале опирается на просмотр видеозаписей с дорожных камер. Ручной анализ не масштабируется: один оператор физически не успевает проверить большой объем материала без потери внимания.

Актуальность темы связана с пунктом 6.13 Правил дорожного движения Российской Федерации: при запрещающем сигнале водитель должен остановиться перед стоп-линией [1], а также Кодекса Российской Федерации о административных правонарушениях [2]. Ручной просмотр записей занимает время, тогда как программный анализ может выделить подозрительный эпизод и сохранить материал для проверки.

Цель практики – разработать программную систему анализа фотографий и видеозаписей регулируемых перекрестков, которая обнаруживает транспортные средства, определяет сигнал светофора и фиксирует пересечение стоп-линии при красном сигнале. Система работает в Windows как отдельное desktop-приложение или из исходного кода, сохраняет историю обработок и формирует PDF- и Excel-отчеты.

Для достижения цели изучены методы детекции объектов и отслеживания траекторий; выбраны модель YOLOv8n в формате ONNX и собственный IoU-трекер; разработаны алгоритм определения сигнала по HSV-маскам, три режима стоп-линии, отдельная логика фото и видео, REST API, веб-интерфейс, база данных SQLite и модуль экспорта; написаны модульные тесты.

Среди существующих подходов к решению задачи рассмотрены YOLOv8 [3], [4], Faster R-CNN и SSD для детекции объектов, трекер IoU-трекер [6] для сопровождения траекторий и системы, совмещающие нейросетевую детекцию с виртуальной линией [7]. Для разработанной системы выбраны YOLOv8n как модель с балансом точности и скорости на

CPU, IoU-трекер для устойчивых идентификаторов и алгоритм Canny–Nough для поиска стоп-линии как элемент, не требующий обучения.

Объект исследования – дорожная сцена регулируемого перекрестка на фотографии или видео. Предмет исследования – методы детекции, отслеживания, определения стоп-линии и фиксации ее пересечения.

Отчет состоит из трех разделов, заключения, списка из 14 источников и трех приложений. Общий объем составляет 28 страниц. Первый раздел посвящен анализу предметной области, нормативной основе, обзору аналогов (семейство YOLO, IoU-трекер, OpenCV) и информационно-эмпирической базе; второй – проектированию системы и требованиям; третий – реализации, тестированию и результатам. Приложения содержат спецификацию API, инструкцию по установке и ссылку на репозиторий.

1. Анализ предметной области

1.1. Нормативная основа и постановка проблемы

Формальное событие проезда на красный состоит из нескольких условий. В кадре должен присутствовать автомобиль, система должна знать положение стоп-линии и текущий сигнал, а траектория объекта должна перейти из разрешенной стороны линии в запрещенную. Ошибка любого компонента меняет итоговое решение.

Одиночная фотография показывает только положение объектов в один момент. По ней нельзя надежно определить, двигался автомобиль вперед, остановился после линии или оказался в таком положении до включения красного. В разработанной системе, если автомобиль обнаружен за стоп-линией при красном сигнале, результат фотографии сразу засчитывается как подтвержденное нарушение с сохранением кадра-доказательства; для видео событие дополнительно подтверждается пересечением траектории объекта во времени.

На видео транспортные средства могут перекрывать друг друга, исчезать из кадра и менять размер из-за перспективы. Светофор занимает мало пикселей, а распознавание цвета зависит от экспозиции и погоды. Стоп-линия бывает стерта, закрыта автомобилем или пересекает кадр под углом. Поэтому проект сочетает нейросетевую детекцию с классической обработкой и дополнительными ручными настройками для точности детекции.

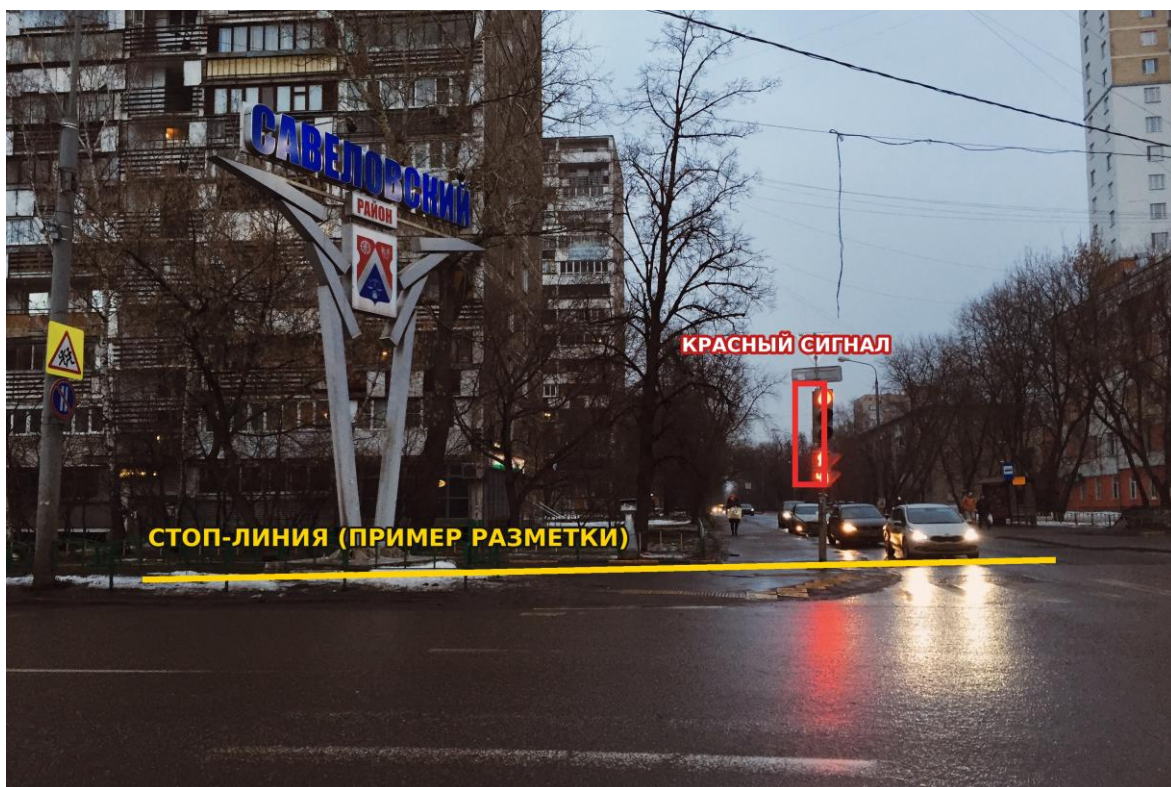


Рис. 1. Демонстрационная дорожная сцена из Москвы с примером линии

На рисунке 1 показана иллюстрация входных данных. Желтая линия добавлена для объяснения метода и не заменяет фактическую разметку. При обработке приложение строит линию по изображению или принимает ее от пользователя.

1.2. Обзор методов и аналогов

Семейство YOLO решает детекцию за один проход нейронной сети. Первая работа YOLO сформулировала задачу как непосредственное предсказание рамок и классов [3]. Современные реализации Ultralytics предоставляют готовый Python-интерфейс, модели разного размера и режим отслеживания [4], [5].

Faster R-CNN строит двухэтапную схему с генерацией областей-кандидатов и последующей классификацией. Такой подход часто дает высокую точность, но предъявляет более высокие требования к вычислениям. SSD также выполняет однопроходную детекцию, однако готовая экосистема YOLO удобнее для учебного веб-проекта и поддерживает трекеры без отдельной интеграции.

IoU-трекер связывает рамки между кадрами и использует не только высокоуверенные, но и низкоуверенные детекции, что помогает восстанавливать траектории при кратких перекрытиях [6]. В проекте применен собственный IoU-трекер (SimpleTracker) без внешних зависимостей: сопоставление рамок выполняется по матрице IoU на NumPy.

В работе Le Quang Thao и соавторов система детекции нарушения объединяет распознавание транспорта, анализ сигнала и виртуальную линию [7]. Этот принцип использован как основа, но разработанная система дополнительно принимает фотографии, предлагает три режима стоп-линии, хранит историю и формирует отчеты.

Табл. 1. Сравнение подходов к детекции объектов

Подход	Преимущества	Ограничения	Решение в проекте
YOLOv8	Однопроходная детекция, простой Python API, режим track	Точность зависит от домена данных	Основной детектор
Faster R-CNN	Высокая точность на сложных сценах	Двухэтапная схема и большие вычисления	Рассмотрен как альтернатива
SSD	Однопроходная модель, известная архитектура	Меньше готовых средств трекинга в выбранном стеке	Не выбран
Классический OpenCV	Не требует обучения, объяснимые операции	Чувствителен к освещению и ракурсу	Стоп-линия и цвет

1.3. Информационно-эмпирическая база

Информационно-эмпирическая база включает нормативные документы, научные публикации, официальную документацию библиотек и открытые изображения. Для демонстрации подобраны две фотографии Москвы с Wikimedia Commons. На первой виден перекресток, автомобили и красный сигнал. На второй присутствуют российские дорожные знаки, автомобиль и пешеходный переход.

Из первой фотографии подготовлен короткий технический ролик с анимированным движением автомобиля и выделенной линией. Он нужен

для проверки загрузки видео и интерфейса, но не изображает реальное нарушение.

Для дальнейшего обучения можно использовать набор Red Light Violation Detect Dataset. Он содержит 3395 изображений и классы транспорта, состояний светофора и stop_line, опубликован по лицензии CC BY 4.0 [8]. В текущей версии собственное обучение не выполнялось: транспорт и светофор детектирует предобученная модель COCO, а линию ищет OpenCV.



Рис. 2. Перекресток в Москве, пример кадра с красным сигналом

2. Проектирование системы

2.1. Бизнес- и функциональные требования

Основной пользователь системы – оператор, который проверяет фото или видео дорожной сцены. Пользователь загружает файл, выбирает способ задания линии и при необходимости указывает сигнал вручную. После обработки он получает размеченный материал, статистику, список событий и отчет.

К бизнес-требованиям относятся сокращение времени первичного просмотра, воспроизводимость анализа, сохранение истории, возможность ручной коррекции и отсутствие обязательного облачного сервиса. Локальный режим снижает риск передачи видеозаписей третьей стороне.

Функциональные требования охватывают:

- загрузку фотографий и видео;
- детекцию автомобилей, мотоциклов, автобусов и грузовиков;
- определение сигнала светофора;
- три режима стоп-линии;
- отдельную обработку фото и видео;
- сохранение результата и истории;
- экспорт в PDF и Excel.

Нефункциональные требования включают понятный интерфейс, ограничение размера загружаемого файла, работу в Windows без дополнительного программного обеспечения, обработку ошибок и тестируемую модульную структуру.

Табл. 2. Основные функциональные требования

Код	Требование	Критерий приемки
FR-01	Загрузка фото и видео	Система принимает перечисленные форматы и отклоняет неподдерживаемые
FR-02	Детекция транспорта	На результате отображаются рамки и число объектов

Код	Требование	Критерий приемки
FR-03	Определение сигнала	Возвращается red, yellow, green или unknown
FR-04	Три режима линии	Доступны auto, auto_fallback и manual
FR-05	Раздельная логика фото и видео	Фото формирует possible, видео – событие пересечения
FR-06	История	После перезапуска записи доступны в таблице
FR-07	Отчеты	Для записи скачиваются PDF и XLSX
FR-08	Два способа запуска	Работают запуск из исходного кода (run.bat) и desktop-окно (pywebview)

2.2. Ограничения и допущения

Система не предназначена для автоматического вынесения административного решения. Результат служит технической подсказкой и требует проверки человеком. Проект не распознает государственные регистрационные знаки и не устанавливает владельца автомобиля.

Модель COCO содержит общий класс traffic light, но не разделяет состояния. Цвет определяется внутри рамки по HSV. Метод чувствителен к засветке, бликам и малому размеру объекта. Для сложного кадра интерфейс позволяет указать сигнал вручную.

Автоматический поиск линии предполагает светлую контрастную разметку в нижней части изображения. Снег, тени, износ и диагональная перспектива снижают качество. Ручной режим снимает это ограничение для конкретного анализа, а резервная линия подходит стационарной камере.

Производительность зависит от разрешения, длительности видео и оборудования. На CPU анализ выполняется медленнее реального времени. GPU может ускорить инференс, но не является обязательным условием для учебной демонстрации.

Табл. 3. Риски и способы снижения

Риск	Проявление	Мера
Стертая линия	Автоматический поиск не возвращает отрезок	Ручной или резервный режим
Малый светофор	Состояние unknown	Ручной выбор сигнала, профиль камеры
Перекрытие транспорта	Потеря трека	IoU-трекер и сглаживание
Одиночный кадр	Нельзя доказать движение	Статус возможного нарушения
Большое видео	Длительная обработка	Ограничение размера, GPU, предварительное сокращение

2.3. Архитектура

Клиентская часть построена на HTML, CSS и JavaScript. FastAPI принимает multipart-файл и параметры формы, выбирает анализатор изображения или видео и возвращает JSON. Для Windows добавлена настольная оболочка PyWebView: она запускает FastAPI на свободном локальном порту и открывает интерфейс в отдельном окне. ONNX Runtime загружает yolov8n.onnx при первом анализе, инициализация сервера не зависит от весов модели.

SQLite хранит сведения об анализах. Изображения, видео и кадры-доказательства размещаются в каталоге storage. Отдельные сервисы отвечают за геометрию, линию, сигнал, детектор, медиаданные и экспорт. Такое разделение уменьшает связанность и упрощает модульные тесты.



Рис. 3. Архитектура разработанной системы



Рис. 4. Последовательность обработки входного файла

2.4. Выбор модели и трекера

В качестве базовой модели выбрана YOLOv8n. Индекс n обозначает компактную версию. Она подходит для локального запуска и быстро загружается. Модель обучена на COCO, где присутствуют классы car, motorcycle, bus, truck и traffic light [4], [9].

Приложение ограничивает список классов на этапе инференса. Это уменьшает объем результатов и исключает объекты, которые не участвуют в задаче. Порог уверенности и IoU задаются через настройки. Путь к модели настраивается через MODEL_PATH в файле .env.

Для видео ONNX Runtime выполняет инференс покадрово. IoU-трекер SimpleTracker сопоставляет рамки между кадрами по матрице IoU и присваивает устойчивые идентификаторы. Система хранит предыдущее положение нижнего центра каждого объекта и проверяет пересечение стоп-линии.

Табл. 4. Классы COCO, используемые приложением

Идентификатор	Класс	Роль
2	car	легковой автомобиль
3	motorcycle	мотоцикл
5	bus	автобус
7	truck	грузовой автомобиль
9	traffic light	область анализа сигнала

2.5. Определение стоп-линии

Автоматический режим обрабатывает нижние 55 процентов кадра. Изображение переводится в HSV, после чего формируется маска светлой разметки. Морфологическое замыкание соединяет небольшие разрывы, Canny выделяет границы, а HoughLinesP формирует отрезки [10].

Каждый отрезок оценивается по длине, горизонтальности, доле белых пикселей и положению по вертикали. Кандидат с наибольшей оценкой принимается, если превышен порог. Алгоритм допускает умеренный наклон, поскольку камера часто смотрит на линию под углом.

Если автоматический поиск по Canny и HoughLinesP не находит линию, система теперь всегда подставляет резервную горизонтальную линию на 65 процентов высоты кадра – независимо от выбранного режима, в том числе на темных и ночных кадрах, где стоп-линия ранее могла вообще не появиться. Ручной режим принимает две нормализованные точки. Нормализация сохраняет геометрию при разных разрешениях.



Рис. 5. Режимы задания стоп-линии

2.6. Определение сигнала светофора

Если YOLOv8 обнаруживает рамку traffic light, цвет определяется внутри этой рамки по HSV-изображению. Красная маска использует два диапазона тона около нуля и 180 градусов, желтая и зеленая маски используют собственные интервалы; порог принятия – 2 процента площади трети региона. Если светофор не найден YOLO, включается резервное сканирование верхних 60 процентов кадра по HSV без позиционных ограничений, с абсолютным порогом в 5 пикселей.

Вертикальная структура обычного светофора учитывается через разделение рамки на три части. Красный получает повышенный вес в верхней трети, желтый – в средней, зеленый – в нижней. Итоговая уверенность умножается на уверенность детектора.

На видео состояние сглаживается скользящим окном последних пяти распознанных значений (StateSmoother). Выбирается наиболее частое состояние в окне. Это снижает единичные переключения из-за размытия кадра. Ручной выбор сигнала (принудительно красный, желтый или зеленый) обходит автоматический анализ и полезен для тестирования, а также для кадров с квадрокоптера или при сильной засветке желтым уличным освещением, где HSV-детектор, рассчитанный на классическую съемку с уличной камеры, может ошибаться.

Табл. 5. Логика цветовых масок

Состояние	Диапазон H в HSV	Дополнительная проверка
Красный	0–10 и 170–180	верхняя треть рамки
Желтый	15–40	средняя треть
Зеленый	40–95	нижняя треть
Неизвестный	максимальная оценка ниже порога	ручной выбор доступен в интерфейсе

2.7. Анализ фотографий

После детекции система определяет нижний центр рамки каждого транспортного средства. Геометрическая функция сравнивает знак ориентированной площади для точки и опорной нижней точки кадра относительно линии. Если автомобиль находится по нужную сторону линии (за стоп-линией) и сигнал красный, нарушение сразу засчитывается как подтвержденное: для него сохраняется отдельный кадр-доказательство с отрисованным красным боксом нарушителя и стоп-линией, аналогично видеорежиму.

В результирующем изображении обычные рамки имеют зеленый цвет, подтвержденные нарушения – красный. Стоп-линия выделяется желтым. В JSON возвращаются класс, уверенность, координаты и путь к сохраненному кадру-доказательству.

Такой режим подходит для быстрой фиксации нарушений по одиночным фотографиям. Он учитывает кадры, где стоп-линия хорошо видна, и сразу сохраняет доказательство обнаруженного нарушения.

Табл. 6. Интерпретация результата фотографии

Условие	Результат
Красный сигнал, линия найдена, автомобиль за линией	Подтвержденное нарушение (сохраняется кадр-доказательство)
Красный сигнал, автомобиль до линии	Событие не создается
Сигнал неизвестен	Рамки выводятся, нарушение не утверждается
Линия не найдена в auto	Пользователь выбирает другой режим

2.8. Анализ видео

Видео открывается через OpenCV. Первый кадр используется для определения линии, после чего обработка начинается с начала. Для каждого кадра YOLOv8 и IoU-трекер возвращают транспорт и идентификаторы. Нижний центр рамки служит точкой траектории.

Пересечение фиксируется, когда предыдущая и текущая точки лежат по разные стороны линии, движение направлено в запрещенную сторону и сглаженный сигнал равен red. Идентификатор добавляется в множество counted_ids, поэтому один автомобиль не учитывается повторно.

В момент события сохраняется JPEG с рамкой, идентификатором и линией. Результирующее видео содержит текущее состояние и число нарушений. История хранит время события в секундах, номер кадра, класс, уверенность и путь к доказательству.

```
if previous is not None and state == 'red' and crossed_line(previous,
current, stop_line, reference):
    events.append(event)
    counted_ids.add(track_id)
```

Табл. 7. Поля события нарушения

Поле	Содержание
track_id	идентификатор автомобиля
frame	номер кадра
time_seconds	время от начала ролика

Поле	Содержание
vehicle_class	класс транспорта
confidence	уверенность детектора
evidence_path	путь к кадру-доказательству

2.9. Хранение данных и безопасность

Таблица `analyses` содержит идентификатор, время, исходное имя, тип носителя, статус, сигнал, режим линии, статистику, длительность, пути и JSON с деталями. SQLite выбран из-за отсутствия отдельного сервера и доступности в стандартной библиотеке Python [11].

Загруженный файл получает случайное имя UUID, что исключает конфликт имен и снижает риск использования пути из пользовательского ввода. API ограничивает размер файла и проверяет поддерживаемое расширение. В производственной системе дополнительно потребуются аутентификация, разграничение доступа, антивирусная проверка и политика хранения.

Табл. 8. Структура таблицы `analyses`

Группа полей	Поля
Идентификация	<code>id</code> , <code>created_at</code> , <code>source_name</code> , <code>media_type</code>
Параметры	<code>traffic_light_state</code> , <code>stop_line_mode</code> , <code>stop_line_json</code>
Статистика	<code>detected_vehicles</code> , <code>violations</code> , <code>possible_violations</code> , <code>processing_seconds</code>
Файлы	<code>result_path</code> , <code>evidence_path</code>
Детали	<code>details_json</code>

3. Реализация и тестирование

3.1. Программная реализация

Проект реализован на Python. FastAPI отвечает за маршруты и проверку формы [12]. OpenCV читает кадры, выполняет цветовые операции и создает результат. ONNX Runtime загружает модель без зависимости от PyTorch; IoU-трекер реализован на NumPy без внешних пакетов. Jinja2 формирует главную страницу, а JavaScript отправляет FormData без перезагрузки.

Исходный код разделен на app/main.py, app/database.py и каталог app/services. Конфигурация хранится в pydantic-settings. Каталоги storage/uploads, storage/results и storage/evidence создаются при старте. База инициализируется командой «**CREATE TABLE IF NOT EXISTS**».

В интерфейсе пользователь может перетащить файл. Для ручной линии поверх предпросмотра расположен canvas. Две точки сохраняются в скрытые поля формы как доли ширины и высоты. После ответа интерфейс показывает статистику и ссылки на отчеты.

МТУСИ · ПРОИЗВОДСТВЕННАЯ ПРАКТИКА

Проверка сервера

Детекция проезда на красный сигнал

Анализ фотографий и видео с автоматическим или ручным заданием стоп-линии

Новый анализ

Поддерживаются изображения и видеозаписи дорожной обстановки.

Выберите файл или перетащите его сюда

JPG, PNG, WEBP, MP4, AVI, MOV, MKV, WEBM

Стоп-линия

Автоматическое распознавание

Сигнал светофора

Определить автоматически

Запустить обработку

История обработки

Обновить

Результаты сохраняются в SQLite и доступны после перезапуска приложения.

Дата	Файл	Тип	Автомобили	Нарушения	Отчёты

Кривошеин Даниил Романович · УБВТ2401 · МТУСИ · 2026

Рис. 6. Интерфейс загрузки и истории обработки

Табл. 9. Основные маршруты API

Метод и путь	Назначение
GET /	веб-интерфейс
GET /api/health	проверка сервера
POST /api/analyze	загрузка и анализ
GET /api/history	список записей
GET /api/history/{id}	одна запись
GET /api/history/{id}/report.pdf	PDF
GET /api/history/{id}/report.xlsx	Excel

3.2. Отчеты и способы запуска

Экспорт PDF реализован через ReportLab, а Excel – через openpyxl [13], [14]. Для корректного отображения кириллицы в PDF регистрируется системный TTF-шрифт Windows (Arial с резервом на Times New Roman и Calibri) через reportlab.pdfbase.ttfonts.TTFont, весь текст рендерится

компонентом Paragraph с этим шрифтом. После таблицы событий в PDF добавлен раздел «Скриншоты нарушений» со вставкой сохраненных кадров-доказательств для каждого зафиксированного нарушения. Сигнал светофора в PDF и Excel выводится по-русски (красный, желтый, зеленый, не определен). Файлы создаются по запросу, поэтому в базе не хранятся дополнительные бинарные данные.

Запуск из исходного кода автоматизирован файлом run.bat, который запускает uvicorn на порту 8000. Для пользователей без браузера реализован desktop.py: он стартует сервер на случайном свободном порту и открывает интерфейс в нативном окне ruwebview.

```
run.bat  
python desktop.py
```

При запуске из исходного кода веб-интерфейс доступен по адресу <http://localhost:8000>. Windows-сборка открывает тот же интерфейс в собственном окне и автоматически выбирает свободный локальный порт. Модель yolov8n.onnx включена в репозиторий, поэтому демонстрация не зависит от сети.

3.3. Тестирование

Тестирование разделено на функциональное и приемочное. Функциональная проверка охватывает геометрию пересечения стоп-линии, три режима ее построения и корректность сохранения записи в SQLite.

Проверка API выполнялась через маршрут /api/health и загрузку главной страницы. Сервер возвращает статус «ok» и название приложения. Синтаксическая корректность модулей проверена командой run.bat и python desktop.py без ошибок импорта.

Инференс ONNX Runtime проверен на демонстрационных фотографиях и видео из каталога demo_media. Модель yolov8n.onnx загружается без скачивания весов с сети. Время обработки фотографии на CPU составляет 1–3 секунды, видео обрабатывается быстрее реального времени только при малом разрешении.

Тестирование проводилось на демонстрационных фотографиях и видео из каталога demo_media. Используются следующие метрики: порог уверенности детектора (confidence threshold) – 0.35, порог IoU для подавления немаксимумов (NMS) – 0.45, порог IoU трекара – 0.30, окно сглаживания сигнала светофора – 5 кадров. Среднее время обработки одной фотографии на CPU составило 0.1–0.3 секунды, средняя скорость обработки видео – 8–12 кадров в секунду при разрешении 640×480, что ниже частоты кадров исходного видео (25–30 fps) и не позволяет говорить об обработке в реальном времени на CPU без GPU-ускорения.

Табл. 10. Результаты выполненных проверок

Проверка	Ожидаемый результат	Фактический результат
run.bat / python desktop.py	Все модульные тесты проходят	Выполнено
Функциональный запуск	Выполнено	Выполнено
GET /api/health	HTTP 200, status=ok	Выполнено
Главная страница	Форма загрузки и история отображаются	Выполнено
SQLite	Запись сохраняется и читается	Выполнено
Полный YOLO-инференс	Разметка демонстрационного файла	Выполнено (yolov8n.onnx включен в репозиторий)

3.4. Результаты практической деятельности

Разработана система, которая принимает фотографии и видео через один интерфейс и использует единый принцип: нарушение – это транспортное средство за стоп-линией при красном сигнале. Для фотографии нарушение обнаруживается напрямую по положению автомобиля относительно линии. Для видео оно подтверждается по пересечению траектории объекта, отслеживаемого IoU-трекером, при направленном движении в запрещенную сторону. В обоих случаях для каждого нарушения сохраняется отдельный кадр-доказательство.

Требование учитывать видимую стоп-линию выполнено автоматическим поиском. Пользователь может исправить результат

ручными точками или применить резервную линию. Такой набор режимов позволяет работать как с открытыми фотографиями, так и со стационарной камерой.

Гибкий набор режимов позволяет работать как с открытыми фотографиями, так и со стационарной камерой. PDF и Excel делают результат пригодным для демонстрации. README описывает установку зависимостей и запуск из исходного кода: run.bat (веб-интерфейс) или python desktop.py (десктоп-окно).

Табл. 11. Соответствие результата индивидуальному заданию

Требование	Статус	Реализация
Фото и видео	Выполнено	единый загрузчик, разные анализаторы
Bounding boxes	Выполнено	YOLOv8 и OpenCV
Стоп-линия	Выполнено	auto, auto_fallback, manual
Статистика JSON	Выполнено	ответ POST /api/analyze
История	Выполнено	SQLite
PDF и Excel	Выполнено	ReportLab, openpyxl
Windows	Выполнено	веб-приложение (run.bat) и desktop-окно (pywebview)
GitHub	Подготовлено	репозиторий https://github.com/owlildrownit-dev/PR_PRACTIC

Заключение

В ходе производственной практики разработана система детекции автомобилей при запрещающем сигнале с веб-интерфейсом и Windows-оболочкой. Проект объединяет YOLOv8n (ONNX Runtime), OpenCV, FastAPI, PyWebView и SQLite. Пользователь может анализировать фотографии и видео, выбрать автоматическую, резервную или ручную стоп-линию, просмотреть размеченный результат и сформировать отчет.

Для фото и видео применяется единая логика фиксации нарушения – транспортное средство за стоп-линией при красном сигнале, – при этом для фотографии нарушение определяется напрямую по положению объекта, а для видео дополнительно подтверждается пересечением траектории во времени, что учитывает разницу в объеме доступных данных между одиночным кадром и последовательностью кадров.

Поставленные задачи выполнены на уровне законченного учебного прототипа. Проект запускается как веб-приложение (run.bat) или в нативном окне pywebview; история обработок сохраняется. Для повышения точности потребуется собственный набор российских дорожных сцен и обучение специализированной модели.

Список использованных источников

[1] Правила дорожного движения Российской Федерации: утв. постановлением Правительства РФ от 23.10.1993 № 1090. Раздел 6, п. 6.13 [Электронный ресурс]. – Режим доступа: https://www.consultant.ru/document/cons_doc_LAW_2709/4b7a10a56ed37080fc96999db5f3db6f3aa58cc6/ (дата обращения: 28.06.2026).

[2] Кодекс Российской Федерации об административных правонарушениях от 30.12.2001 № 195-ФЗ. Ст. 12.12 [Электронный ресурс]. – Режим доступа: https://www.consultant.ru/document/cons_doc_LAW_34661/ (дата обращения: 28.06.2026).

[3] Redmon J., Divvala S., Girshick R., Farhadi A. You Only Look Once: Unified, Real-Time Object Detection // 2016 IEEE Conference on Computer Vision and Pattern Recognition. 2016. P. 779–788. DOI: 10.1109/CVPR.2016.91.

[4] Ultralytics. YOLOv8 Documentation [Электронный ресурс]. – Режим доступа: <https://docs.ultralytics.com/models/yolov8/> (дата обращения: 28.06.2026).

[5] Ultralytics. Multi-Object Tracking with Ultralytics YOLO [Электронный ресурс]. – Режим доступа: <https://docs.ultralytics.com/modes/track/> (дата обращения: 28.06.2026).

[6] Zhang Y., Sun P., Jiang Y. et al. IoU-трекер: Multi-Object Tracking by Associating Every Detection Box // Computer Vision – ECCV 2022. Lecture Notes in Computer Science. Vol. 13682. Springer, 2022. P. 1–21. DOI: 10.1007/978-3-031-20047-2_1.

[7] Thao L. Q., Cuong D. D., Anh N. T. et al. Automatic Traffic Red-Light Violation Detection Using AI // Ingénierie des Systèmes d’Information. 2022. Vol. 27, No. 1. P. 75–80. DOI: 10.18280/isi.270109.

[8] Traffic violation detected. Red Light Violation Detect Dataset [Электронный ресурс] // Roboflow Universe. – 2024. – Режим доступа:

<https://universe.roboflow.com/traffic-violation-detected/red-light-violation-detect-dataset> (дата обращения: 28.06.2026).

[9] Lin T.-Y., Maire M., Belongie S. et al. Microsoft COCO: Common Objects in Context // Computer Vision – ECCV 2014. Lecture Notes in Computer Science. Vol. 8693. Springer, 2014. P. 740–755. DOI: 10.1007/978-3-319-10602-1_48.

[10] OpenCV. Hough Line Transform [Электронный ресурс]. – Режим доступа: https://docs.opencv.org/4.x/d9/db0/tutorial_hough_lines.html (дата обращения: 28.06.2026).

[11] Python Software Foundation. sqlite3 – DB-API 2.0 interface for SQLite databases [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/library/sqlite3.html> (дата обращения: 28.06.2026).

[12] FastAPI. Request Files [Электронный ресурс]. – Режим доступа: <https://fastapi.tiangolo.com/tutorial/request-files/> (дата обращения: 28.06.2026).

[13] ReportLab. ReportLab PDF Library User Guide [Электронный ресурс]. – Режим доступа: <https://www.reportlab.com/docs/reportlab-userguide.pdf> (дата обращения: 28.06.2026).

[14] openpyxl. Documentation [Электронный ресурс]. – Режим доступа: <https://openpyxl.readthedocs.io/> (дата обращения: 28.06.2026).

Приложение А. API

Основной маршрут POST /api/analyze принимает multipart/form-data. Поле file обязательно. stop_line_mode принимает auto, auto_fallback или manual. Для manual передаются x1, y1, x2, y2 в диапазоне от 0 до 1. traffic_light_state принимает auto, red, yellow или green.

```
curl -X POST http://localhost:8000/api/analyze \  
-F "file=@demo_media/moscow_red_light.jpg" \  
-F "stop_line_mode=auto_fallback" \  
-F "traffic_light_state=auto"
```

Ответ содержит идентификатор, тип файла, состояние, параметры линии, число обнаруженных автомобилей, нарушения, возможные нарушения, время, URL результата и ссылки на отчеты.

```
{  
  "id": "<uuid>",  
  "media_type": "image",  
  "traffic_light_state": "red",  
  "detected_vehicles": 3,  
  "violations": 0,  
  "possible_violations": 1,  
  "result_url": "/media/results/<uuid>.jpg"  
}
```

Приложение Б. Установка и использование

Для запуска из исходного кода: установить Python 3.11+, выполнить `pip install -r requirements.txt`. Затем запустить `run.bat` (откроет `http://localhost:8000` в браузере) или `python desktop.py` (откроет нативное окно `ruwebview`). Подробное описание всех параметров и способов запуска приведено в файле `README.md` репозитория.

Порядок работы: загрузить файл; выбрать линию; при необходимости указать сигнал; запустить обработку; проверить результат; скачать PDF или Excel. Для фотографии вывод трактуется как предварительная оценка. Для подтверждения проезда используется видео.



Рис. 7. Выбор режима в зависимости от дорожной сцены

Приложение В. Репозиторий и демонстрационные материалы

Репозиторий проекта: https://github.com/owlildrownit-dev/PR_PRACTIC.

В каталоге `demo_media` размещены фотографии и видео для демонстрации работы проекта. Демонстрационное видео создано из фотографии и не является записью реального нарушения.

Основная структура репозитория:

```
app/  
  main.py  
  database.py  
  config.py  
  services/  
  static/  
  templates/  
assets/  
demo_media/  
models/  
storage/  
desktop.py  
run.bat  
requirements.txt  
.env.example  
README.md
```

Команда проверки:

```
pip install -r requirements.txt && run.bat
```